

Fig. 8: Compilation window

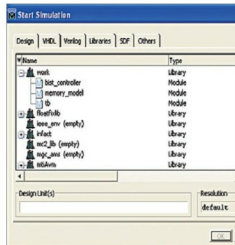


Fig. 9: Start Simulation window

Start on Welcome screen.

3. Create Project window pops up (Fig. 2). Select a suitable name for your project. Set Project Location to C:/Documents and Settings/Nidhi/Desktop/Final_BIST (in our case) and leave the rest as default, followed by clicking OK.

4. Add Items to the Project window pop up (Fig. 3).

5. On this window, select Create New File option.

6. Create Project File window pops up. Give an appropriate file name (say, `memory_model.v`) for the file you want to add, and choose Verilog as Add File as Type and Top Level as Folder (Fig. 4).

7. On the workspace section of the main window (Fig. 5), double-click on the file you have created (memory model.v in our case).

8. Type your Verilog code (`memory_model.v`) in the new window. Here we have considered 1024 inputs, so we write first data as `32'h42_40_40_40`. Note that, these inputs are arbitrary and so you can enter any 32-bit data. Our goal is to obtain the same data back. Here, we test the memory manually and verify the BIST operation of a memory model using

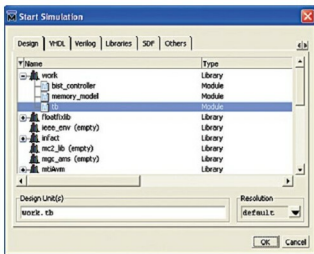


Fig. 10: Library tab

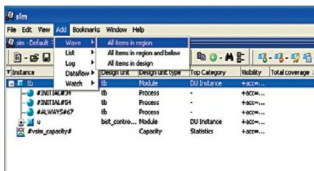


Fig. 11: Add wave to the project

test bench and BIST controller.

In this example, we first write data as 1s in all memory locations. Then we read back all locations. We expect all the read data to be 1s in the result. If there is even a single 0 in the result, that means some cell or bit is stuck-at fault 0. There exists a fault and so BIST test has failed. This is achieved by ANDING all bits.

9. Save your code from File menu.

10. Add relevant Verilog source files that actually test the BIST operation of a memory. These include memory model, BIST controller and a test bench, which will then check the output for each input.

Add new files to BIST project by right-clicking memory model.v file.

Select Add to Project→New File option as shown in Fig. 6.

Give File Name bist_controller.v and follow the steps from 7 through 9 as mentioned above.

Similarly, add test bench file (tb.v) to the project and enter re-

spective Verilog codes in these files.

The test bench should indicate bist_status 0 and bist_status 1 for BIST passed and BIST failed, respectively.

The final workspace window is shown in Fig. 7.

Compiling/debugging project files.

1. Select Compile→
Compile All option.

2. Compilation result is shown on the main window. A green tick is shown against each file name, which means there are no errors in the project (Fig. 8).

Simulating BIST design

1. Click on Library menu from the main window and click on

plus (+) sign next to the work library. You should see `memory_model`, `bist_controller` and `tb` that we have just compiled (Fig. 9).

2. Now, in the work library, select **tb** and click **OK** (Fig. 10). This will open **sim** default window as shown in Fig. 11.

3. Go to Add→To Wave→All items in design options.

4. Select the signals that you want to monitor for simulation purposes. This is a test bench, so it will write something using the pattern generator and read back the same pattern automatically.

This is the main concept of BIST. No external inputs are required when the memory model is in BIST mode.

We are now ready to simulate by writing `do sim.do` command in the transcript window (Fig. 12). Now, click Enter; you should be able to see the output being displayed on the console (transcript window). BIST captures the pass or fail status